TODO: Paper Title

Anonymous author
Anonymous affiliation

Abstract

Background. TODO: 1–2 sentences on the problem and why it matters.
Aims. TODO: the research questions or specific objectives.
Method. TODO: dataset, models, approaches compared.
Results. TODO: headline numbers, ordered by RQ.
Conclusions. TODO: what this means and the takeaway for practitioners.

2012 ACM Subject Classification Software and its engineering → Software notations and tools; Computing methodologies → Natural language processing

Keywords and phrases issue classification, retrieval-augmented generation, fine-tuning, large language models, mining software repositories

Digital Object Identifier 10.4230/LIPIcs...

1 Introduction

2 Related Work

Early IRC studies employed data mining techniques and ML classifiers [1, 18, 12]. For example, Antoniol et al. [1] leveraged a naive Bayes classifier to distinguish bugs from other issue reports. However, these methods showed low performance due to their limited semantic understanding of the issue report text [13].

To overcome this, recent methods have adopted transformer models due to their strong contextual understanding [6, 4, 24, 16, 17]. These studies typically fine-tune the models on historical issues in a supervised setting. For instance, Izadi et al. [16] fine-tuned a RoBERTa [21] model to classify issues into *Bugs*, *Features*, or *Questions*. Similarly, Trautsch et al. [24] trained a seBERT [25] model to classify issues into the same three categories. Although transformer-based approaches achieve strong results, they rely on large amounts of project-specific data to be effective, hindering their generalizability on projects with a small number of labeled issues [13, 16]. Additionally, they struggle to accurately classify minority classes with fewer training examples [17, 6].

These challenges have motivated the exploration of LLMs, which exhibit strong text classification capabilities thanks to extensive pretraining on massive corpora. [13, 3, 2, 5, 26, 8]. Studies show that LLMs perform well in zero-shot settings [8, 2]. SOTA LLM-based methods achieved the best performance by fine-tuning the models on labeled datasets [13, 3]. For example, Aracena et al. [3] instruction-tuned several models on the NLBSE '23 and '24 datasets.

3 Approach

3.1 Problem Formulation

Given a query issue report, we classify it into only one of the three labels: *bug*, *feature*, or *question*. Only the issue's title and description text are used for classification with no other signals from the issue tracker. The defined label space is adopted directly from prior IRC studies [2, 3, 16, 18, 1].

© Anonymous author(s);
licensed under Creative Commons License CC-BY 4.0
Leibniz International Proceedings in Informatics
LIPICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

3.2 Indexing and Retrieval

To prepare the issues for retrieval based on textual similarity, we first embed their textual content (i.e., *title* + *description*). We do not embed the issue labels. Instead, we retain them as metadata associated with each embedded issue. To generate the embeddings, we use the `all-MiniLM-L6-v2` model from the Sentence-Transformers library [22]. We selected this model due to its lightweight architecture, fast performance, and effectiveness on issue report text [15]. The embeddings are then indexed using the Faiss vector database library [10], which provides efficient high-dimensional similarity retrieval. At the retrieval stage, the query issue is first embedded with the same model. Next, Faiss calculates the cosine similarity of indexed issue reports to the query, and returns the top-K nearest neighbors, ordered by descending similarity.

3.3 RAGTAG

RAGTAG is our proposed retrieval-augmented few-shot prompting approach for IRC. RAGTAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors to the query issue, and present them as classified examples to the LLM in a few-shot prompt. This enables a more accurate classification, based on in-context learning [5]. The choice to present the top-K nearest neighbors as examples is based on two insights from prior research: First, retrieval-based example selection outperforms random selection [19]. Second, the most similar neighbors tend to share the same labels [9]; therefore, they are informative for the classification.

We create the few-shot prompt in three parts. First, the system message frames the classification task and the label space. Second, the top-K nearest neighbors are listed in retrieval order (descending similarity), each formatted as *title* + *description* + *label*. Finally, the query issue is appended to the end of the prompt, ensuring it receives high attention from the LLM [20]. $K = 0$ produces a zero-shot prompting baseline, similar to prior work [7].

We evaluate RAGTAG at top-K values in $\{0, 1, 3, 6, 9\}$. Details for the K value grid selection are provided in Section 5.1. Since the median prompt size at $K = 9$ in our dataset is ∼6,300 tokens, the maximum sequence length is set to 8,192 tokens. Roughly 30% of the context quota is reserved for the query issue and 70% for the neighbors. If the neighbors exceed their overall quota, they are truncated proportionally based on their length (longer neighbors get more truncation). The query issue gets truncated if it exceeds its budget as well. Truncation happens on the right side of the description text, preserving all titles and labels (if any).

The LLM is instructed to generate the predicted label wrapped in XML tags (e.g. `<label>bug</label>`). The labels of the presented examples also follow the same format. This technique enables easy detection of the predicted label from the LLM's generation, and reduces variance in LLM outputs caused by prompt format sensitivity [23]. We use a regular expression to extract the predicted label. Unparseable outputs are marked as *invalid*.

3.4 VOTAG

VOTAG is our proposed non-parametric retrieval IRC method. VOTAG uses the retrieval pipeline described in Section 3.2 to retrieve the top-K nearest neighbors of the query issue report, then predicts the final label with a similarity-weighted vote on their labels. This approach follows the standard K-nearest-neighbor classification setup [9], with neighbors

weighted by their similarity to the query [11].[1] Formally, for a query issue q with retrieved neighbors $\{(n_i, l_i, s_i)\}_{i=1}^{K}$, where n_i is the i-th nearest neighbor with label l_i and cosine similarity s_i to q, the label is predicted by:

$$\hat{y} = \arg\max_{c \in \mathcal{L}} \sum_{i=1}^{K} s_i \cdot \mathbf{1}[l_i = c], \tag{1}$$

where $\mathcal{L}$ is the label set and $\mathbf{1}[\cdot]$ is the indicator function. VOTAG breaks ties by returning the label of the most similar neighbor in the tied classes.

VOTAG has three main roles: First, it serves as a fast and cost-efficient baseline that LLM-based approaches must clear to justify their computational cost. Second, it is used as an ablation to distinguish the contributions of the LLM and retrieval in the RAGTAG method. Third, VOTAG's performance as a function of K informs the K value grid evaluated for RAGTAG.

3.5 Fine-Tuning

SOTA IRC approaches have achieved strong performance by supervised fine-tuning of LLMs [13, 3, 2], using Low-Rank Adaptation (LoRA) [14]. In these studies, the model is trained on labeled issue reports and predicts the labels for unseen test issues. Each training example is formatted as an instruction prompt which includes the issue's title and description as input, and its label as output. At inference, the trained model receives the test issue in the same format, and predicts the empty label field. Regular expressions are used to extract the prediction from the output, and unparseable outputs are marked as *invalid*. We use this established methodology as our fine-tuning baseline.

For memory efficiency, each model is loaded in 4-bit quantization with Unsloth[2]. The LoRA rank and alpha are set to 16, with a learning rate of 2×10^{-4} and the paged AdamW 8-bit optimizer. Training runs for 3 epochs with a per-device batch size of 1 and gradient accumulation steps of 16. The max sequence length is set to 2,048 tokens, which covers 94.9% of the issues in our dataset. Issues that exceed this budget get truncated from the right side of their description text to fit.

4 Experimental Setup

4.1 Dataset and Settings

We use the dataset introduced in Heo et al. [13] for our experiments. The dataset has a total of 6,600 issue reports gathered from eleven active open-source projects: `ansible/ansible`, `bitcoin/bitcoin`, `dart-lang/sdk`, `dotnet/roslyn`, `facebook/react`, `flutter/flutter`, `kubernetes/kubernetes`, `microsoft/TypeScript`, `microsoft/vscode`, `opencv/opencv`, and `tensorflow/tensorflow`. Each project has 600 issue reports with an equal distribution across the labels (*bug, feature, question*). To support their LLM fine-tuning method, Heo et al. divided the dataset into 50/50 train and test splits, stratified by project and label. This results in 300 train and 300 test issue reports per project.

We also adopt Heo et al.'s project-specific (PS) and project-agnostic (PA) configurations to examine how data scope affects IRC performance. In the PS setting, each project's train

[1] Squared-similarity weighting and uniform majority were also evaluated, but were outperformed by similarity-weighted voting.

[2] `https://unsloth.ai`

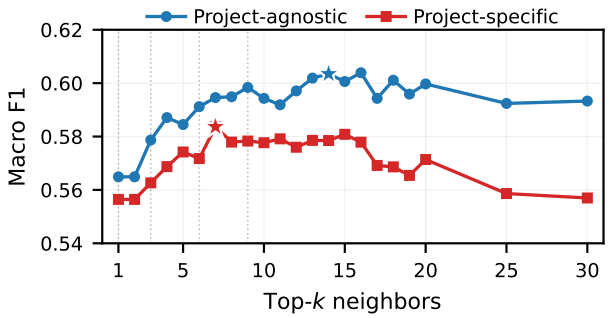

Figure 1 VOTAG macro F1 as a function of k.

and test data are used in isolation, resulting in eleven different train-test pairs. In the PA setting, the train splits of all projects are merged into a single training set of 3,300 issue reports. In both configurations, RAGTAG and VOTAG only use the training splits as their retrieval index to ensure fair comparison with the fine-tuning baseline.

4.2 Models

We use the Qwen2.5-Instruct model family [27] in our evaluations with four different parameter sizes: 3B, 7B, 14B, and 32B. Qwen2.5 is an open-source LLM with strong performance on general-purpose benchmarks. Using the same model at different parameter sizes isolates the effect of model scale from model architecture across LLM-based IRC approaches. Model temperature is set at 0.1 for deterministic classifications.

4.3 Evaluation Metrics

4.4 Setup Hardware

5 Evaluations

5.1 RQ1) To what extent can similarity-weighted voting on the nearest neighbors (VOTAG) classify issue reports, and at what point does adding neighbors stop helping?

To study VOTAG's performance as a function of K, We evaluate it at K values 1 to 30 across both PS and PA settings. Figure 1 plots macro F1 as a function of k in both settings, while Tables 1 and 2 report VOTAG's peak per-class and macro F1 at the chosen plateau k (k=7 for PS, k=14 for PA, leftmost column of each table).

5.2 Bug-Bias Diagnosis

6 Discussion

7 Threats to Validity

8 Conclusion

9 Data Availability

References

1. Giuliano Antoniol, Kamel Ayari, Massimiliano Di Penta, Foutse Khomh, and Yann-Gaël Guéhéneuc. Is it a bug or an enhancement? a text-based approach to classify change requests. In *Proceedings of the 2008 conference of the center for advanced studies on collaborative research: meeting of minds*, pages 304–318, 2008.
2. Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A. Gerosa. Applying large language models api to issue classification problem, 2024. URL: `https://arxiv.org/abs/2401.04637`, `arXiv:2401.04637`.
3. Gabriel Aracena, Kyle Luster, Fabio Santos, Igor Steinmacher, and Marco A Gerosa. Applying large language models to issue classification: Revisiting with extended data and new models. *Science of Computer Programming*, page 103333, 2025.
4. Shikhar Bharadwaj and Tushar Kadam. Github issue classification using bert-style models. In *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, pages 40–43, 2022.
5. Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
6. Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Issue report classification using pre-trained language models. In *Proceedings of the 1st International Workshop on Natural Language-based Software Engineering*, pages 29–32, 2022.
7. Giuseppe Colavito, Filippo Lanubile, and Nicole Novielli. Benchmarking large language models for automated labeling: The case of issue report classification. *Information and Software Technology*, page 107758, 2025.
8. Giuseppe Colavito, Filippo Lanubile, Nicole Novielli, and Luigi Quaranta. Leveraging gpt-like llms to automate issue labeling. In *Proceedings of the 21st International Conference on Mining Software Repositories*, pages 469–480, 2024.
9. Thomas Cover and Peter Hart. Nearest neighbor pattern classification. *IEEE transactions on information theory*, 13(1):21–27, 1967.
10. Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE Transactions on Big Data*, 2025.
11. Sahibsingh A Dudani. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics*, (4):325–327, 1976.
12. Qiang Fan, Yue Yu, Gang Yin, Tao Wang, and Huaimin Wang. Where is the road for issue reports classification based on text mining? In *2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*, pages 121–130. IEEE, 2017.
13. Jueun Heo and Seonah Lee. A study on applying large language models to issue classification. In *2025 IEEE/ACM 33rd International Conference on Program Comprehension (ICPC)*, pages 1–11. IEEE Computer Society, 2025.

14 Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

15 Huihui Huang, Ratnadira Widyasari, Ting Zhang, Ivana Clairine Irsan, Jieke Shi, Han Wei Ang, Frank Liauw, Eng Lieh Ouh, Lwin Khin Shar, Hong Jin Kang, et al. Back to the basics: Rethinking issue-commit linking with llm-assisted retrieval. *arXiv preprint arXiv:2507.09199*, 2025.

16 Maliheh Izadi. Catiss: An intelligent tool for categorizing issues reports using transformers. In *Proceedings of the 1st international workshop on natural language-based software engineering*, pages 44–47, 2022.

17 Maliheh Izadi, Kiana Akbari, and Abbas Heydarnoori. Predicting the objective and priority of issue reports in software repositories. *Empirical Software Engineering*, 27(2):50, 2022.

18 Rafael Kallis, Andrea Di Sorbo, Gerardo Canfora, and Sebastiano Panichella. Ticket tagger: Machine learning driven issue classification. In *2019 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, pages 406–409. IEEE, 2019.

19 Jiachang Liu, Dinghan Shen, Yizhe Zhang, William B Dolan, Lawrence Carin, and Weizhu Chen. What makes good in-context examples for gpt-3? In *Proceedings of Deep Learning Inside Out (DeeLIO 2022): The 3rd workshop on knowledge extraction and integration for deep learning architectures*, pages 100–114, 2022.

20 Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics*, 12:157–173, 2024.

21 Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. Roberta: A robustly optimized bert pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

22 Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. *arXiv preprint arXiv:1908.10084*, 2019.

23 Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How i learned to start worrying about prompt formatting. *arXiv preprint arXiv:2310.11324*, 2023.

24 Alexander Trautsch and Steffen Herbold. Predicting issue types with sebert. In *Proceedings of the 1st international workshop on natural language-based software engineering*, pages 37–39, 2022.

25 Julian Von der Mosel, Alexander Trautsch, and Steffen Herbold. On the validity of pre-trained transformers for natural language processing in the software engineering domain. *IEEE Transactions on Software Engineering*, 49(4):1487–1507, 2022.

26 Jason Wei, Maarten Bosma, Vincent Y Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M Dai, and Quoc V Le. Finetuned language models are zero-shot learners. *arXiv preprint arXiv:2109.01652*, 2021.

27 An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

Table 1 Per-class F1 score for each project under the project-specific setting, across all four Qwen2.5-Instruct sizes (3B/7B/14B/32B). VOTAG is model-independent (k=7 plateau). RAGTAG uses the best k per scale (k=3 for 3B, k=6 for 7B, k=9 for 14B/32B). FT for 14B/32B is currently a 1-epoch baseline pending the in-flight 3-epoch update.

		VOTAG	**Zero-shot**				**RAGTAG**				**Fine-tune**			
Project	Label	—	3B	7B	14B	32B	3B	7B	14B	32B	3B	7B	14B	32B
ansible	bug	0.644	0.751	0.722	0.760	0.800	0.786	0.813	0.817	0.828	0.943	0.736	0.828	0.911
	feature	0.667	0.875	0.708	0.744	0.727	0.790	0.774	0.824	0.850	0.949	0.933	0.783	0.817
	question	0.397	0.514	0.339	0.198	0.246	0.439	0.478	0.384	0.474	0.910	0.706	0.695	0.728
	average	0.569	0.713	0.590	0.567	0.591	0.672	0.688	0.675	0.717	0.934	0.792	0.768	0.819
bitcoin	bug	0.558	0.696	0.700	0.692	0.702	0.684	0.699	0.700	0.709	0.692	0.600	0.643	0.663
	feature	0.693	0.695	0.844	0.867	0.860	0.874	0.873	0.891	0.872	0.839	0.736	0.678	0.826
	question	0.482	0.313	0.424	0.339	0.440	0.466	0.570	0.507	0.599	0.479	0.605	0.468	0.607
	average	0.577	0.568	0.656	0.632	0.667	0.675	0.714	0.699	0.727	0.670	0.647	0.596	0.699
dart-lang	bug	0.706	0.749	0.766	0.765	0.814	0.782	0.800	0.815	0.847	0.808	0.822	0.693	0.857
	feature	0.639	0.662	0.815	0.835	0.878	0.816	0.832	0.889	0.878	0.788	0.831	0.756	0.910
	question	0.471	0.687	0.762	0.712	0.798	0.715	0.772	0.818	0.856	0.794	0.827	0.719	0.863
	average	0.605	0.699	0.781	0.771	0.830	0.771	0.801	0.841	0.860	0.796	0.827	0.723	0.877
dotnet	bug	0.568	0.652	0.659	0.654	0.643	0.628	0.659	0.645	0.658	0.617	0.585	0.602	0.513
	feature	0.628	0.604	0.781	0.808	0.791	0.756	0.796	0.782	0.806	0.772	0.757	0.458	0.737
	question	0.356	0.326	0.303	0.290	0.397	0.412	0.406	0.406	0.532	0.296	0.468	0.513	0.632
	average	0.517	0.527	0.581	0.584	0.610	0.599	0.620	0.611	0.665	0.562	0.603	0.524	0.628
react	bug	0.783	0.763	0.766	0.775	0.810	0.797	0.797	0.803	0.833	0.886	0.632	0.785	0.804
	feature	0.491	0.728	0.827	0.872	0.870	0.802	0.827	0.871	0.871	0.773	0.813	0.696	0.835
	question	0.534	0.670	0.691	0.632	0.703	0.623	0.687	0.718	0.762	0.721	0.618	0.673	0.724
	average	0.603	0.721	0.761	0.759	0.794	0.741	0.770	0.797	0.822	0.793	0.687	0.718	0.787
flutter	bug	0.659	0.692	0.719	0.701	0.714	0.695	0.700	0.647	0.687	0.720	0.670	0.722	0.701
	feature	0.560	0.817	0.861	0.842	0.842	0.834	0.883	0.857	0.870	0.787	0.888	0.737	0.860
	question	0.483	0.532	0.604	0.607	0.658	0.551	0.684	0.662	0.765	0.419	0.646	0.615	0.715
	average	0.568	0.681	0.728	0.716	0.738	0.694	0.755	0.722	0.774	0.642	0.734	0.691	0.759
kubernetes	bug	0.626	0.659	0.625	0.586	0.652	0.778	0.792	0.817	0.837	0.639	0.783	0.664	0.503
	feature	0.708	0.953	0.850	0.821	0.851	0.904	0.918	0.939	0.929	0.860	0.791	0.911	0.802
	question	0.792	0.459	0.587	0.493	0.627	0.819	0.741	0.718	0.763	0.433	0.727	0.606	0.682
	average	0.709	0.690	0.687	0.633	0.710	0.833	0.817	0.824	0.843	0.644	0.767	0.727	0.662
typescript	bug	0.462	0.660	0.664	0.671	0.683	0.588	0.682	0.654	0.661	0.641	0.593	0.629	0.500
	feature	0.667	0.675	0.749	0.744	0.764	0.712	0.718	0.756	0.765	0.500	0.615	0.239	0.729
	question	0.512	0.292	0.298	0.214	0.341	0.488	0.394	0.417	0.531	0.301	0.490	0.433	0.582
	average	0.547	0.542	0.570	0.543	0.596	0.596	0.598	0.609	0.652	0.481	0.566	0.434	0.604
vscode	bug	0.604	0.590	0.659	0.618	0.617	0.624	0.655	0.664	0.715	0.580	0.575	0.613	0.617
	feature	0.548	0.548	0.785	0.737	0.751	0.776	0.753	0.754	0.786	0.697	0.761	0.660	0.758
	question	0.736	0.137	0.271	0.144	0.177	0.564	0.496	0.558	0.729	0.174	0.638	0.398	0.383
	average	0.629	0.425	0.572	0.500	0.515	0.655	0.635	0.658	0.744	0.484	0.658	0.557	0.586
opencv	bug	0.584	0.620	0.627	0.625	0.664	0.652	0.670	0.675	0.736	0.641	0.540	0.604	0.519
	feature	0.594	0.599	0.800	0.794	0.822	0.794	0.815	0.823	0.847	0.756	0.675	0.542	0.716
	question	0.650	0.320	0.434	0.406	0.543	0.580	0.596	0.586	0.709	0.358	0.506	0.409	0.659
	average	0.610	0.513	0.620	0.608	0.676	0.675	0.694	0.695	0.764	0.585	0.574	0.518	0.631
tensorflow	bug	0.619	0.680	0.755	0.760	0.792	0.782	0.789	0.773	0.789	0.793	0.549	0.830	0.886
	feature	0.385	0.679	0.667	0.817	0.833	0.766	0.793	0.777	0.776	0.798	0.638	0.709	0.779
	question	0.456	0.471	0.574	0.538	0.644	0.621	0.708	0.699	0.741	0.589	0.586	0.697	0.712
	average	0.487	0.610	0.665	0.705	0.756	0.723	0.764	0.750	0.769	0.727	0.591	0.745	0.792
Overall	*average*	0.584	0.608	0.656	0.638	0.680	0.694	0.714	0.717	0.758	0.665	0.677	0.637	0.713

Table 2 Per-class F1 score under the project-agnostic setting, across all four Qwen2.5-Instruct sizes. VOTAG uses k=14 (PA plateau). RAGTAG uses the best k per scale.

	VOTAG	**Zero-shot**				**RAGTAG**				**Fine-tune**			
Label	—	3B	7B	14B	32B	3B	7B	14B	32B	3B	7B	14B	32B
bug	0.635	0.682	0.696	0.691	0.715	0.709	0.729	0.727	0.758	0.736	0.762	0.703	0.778
feature	0.626	0.723	0.791	0.806	0.814	0.802	0.808	0.827	0.838	0.790	0.846	0.768	0.814
question	0.549	0.435	0.500	0.439	0.534	0.572	0.600	0.597	0.683	0.599	0.678	0.676	0.647
average	0.603	0.613	0.662	0.645	0.688	0.694	0.712	0.717	0.759	0.708	0.762	0.715	0.746